

CHAPTER FOUR

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Objectives of the Design

The design of the interactive legal document summarization system is guided directly by the aim and objectives stated in Chapter One. The overall objective of the design is to translate the three-tier architecture and the use case model presented in Chapter Three into a working system that allows a student or graduate with no legal training to upload a contract document and receive a structured, plain-language explanation of its key clauses. The specific design objectives are as follows.

First, the design seeks to provide a simple and accessible upload mechanism that accepts contract documents in PDF or DOCX format without requiring the user to understand the underlying file-processing steps. Second, the design seeks to structure the Natural Language Processing pipeline so that clause identification and plain-language summary generation occur as a single continuous process from the user's perspective, even though multiple internal modules are involved. Third, the design seeks to present the output of this pipeline through an interface that supports both linear reading and visual, at-a-glance comprehension, in line with Nielsen's (1994) usability heuristics discussed in Chapter Two. Fourth, the design seeks to secure user data and uploaded documents through authenticated, token-based access so that a user's contracts remain private. Fifth, the design seeks to ensure that each module of the system, from file intake to visualization, can be developed, tested and refined independently, consistent with the Agile methodology adopted in Chapter Three.

4.2 Control Centre/Main Menu

The control centre of the system is the Dashboard, which serves as the main menu presented to an authenticated user immediately after login. The Dashboard is the central navigational hub from which every other function of the system is reached, and it was designed to minimise the number of steps required for a first-time user to upload and review a contract.

The Dashboard presents four primary navigational elements: an Upload Contract action, which opens the document upload interface; a Summary History panel, which lists previously uploaded and processed documents; a Visualization Dashboard link, which opens the timeline and clause-distribution views for a selected document; and an Account section, which allows the user to manage login credentials. This arrangement follows the principle of recognition rather than recall (Nielsen, 1994), ensuring that a returning user can locate any prior document without needing to remember file names or upload dates, since each entry in the Summary History panel is labelled with the document title, upload date and processing status.

4.3 The Submenus/Subsystems

Beneath the main Dashboard, the system is organised into four functional subsystems, each corresponding to a use case identified in the Use Case Diagram of Chapter Three (Figure 3.2).

4.3.1 Authentication Subsystem

This subsystem handles user registration, login and session management. It is implemented using Django REST Framework SimpleJWT and issues access and refresh tokens on successful login. All subsequent requests from the presentation layer include the access token in the request header, and the Authentication Subsystem validates this token before granting access to any document or summary belonging to the requesting user.

4.3.2 Document Upload and Extraction Subsystem

This subsystem accepts a contract file in PDF or DOCX format, validates the file type and size, and extracts raw text using PyPDF2 for PDF files and python-docx for DOCX files. The extracted text is passed to the NLP subsystem for processing and is also stored, together with the file metadata, in the PostgreSQL database.

4.3.3 NLP Processing Subsystem

This subsystem receives extracted contract text and performs two sequential operations: text preprocessing and clause identification using spaCy, followed by abstractive summarization of each identified clause using Hugging Face Transformers. The output of this subsystem is a structured set of clause objects, each containing the original clause text, its plain-language summary, and its clause type label (for example, termination, payment or confidentiality).

4.3.4 Visualization and Review Subsystem

This subsystem renders the processed clause objects using Recharts, producing a timeline view of contract duration and renewal dates, summary cards for each clause type, and a side-by-side comparison view of original and simplified clause text. It also implements the search and filter function that allows the user to locate a specific clause type within a processed document.

4.4 System Specifications

4.4.1 Database Development Tool

PostgreSQL was selected as the database management system for this project. PostgreSQL was chosen because it offers strong support for relational data with well-defined foreign key constraints, native support for JSON fields, which is useful for storing variable-length clause and summary data, and reliable integration with the Django Object-Relational Mapper (ORM) used in the application layer.

4.4.2 Database Design and Structure

The database is organised around five core tables: User, Document, Clause, Summary and EvaluationLog. The User table stores account credentials and authentication metadata. The Document table stores metadata for each uploaded contract, including the file name, upload date, file type and a foreign key reference to the uploading User. The Clause table stores each clause identified within a document, including its clause type label and its position within the source document, and is linked to the Document table through a foreign key. The Summary table stores the plain-language summary generated for each clause and is linked to the Clause table. The EvaluationLog table records usability and comprehension evaluation data collected during user testing, described further in Chapter Five, and is linked to both the User and Document tables.

The relationships between these tables form a one-to-many structure at each level: one User may own many Documents, one Document may contain many Clauses, and one Clause has exactly one corresponding Summary. This structure is illustrated in the Entity Relationship Diagram in Figure 4.1.

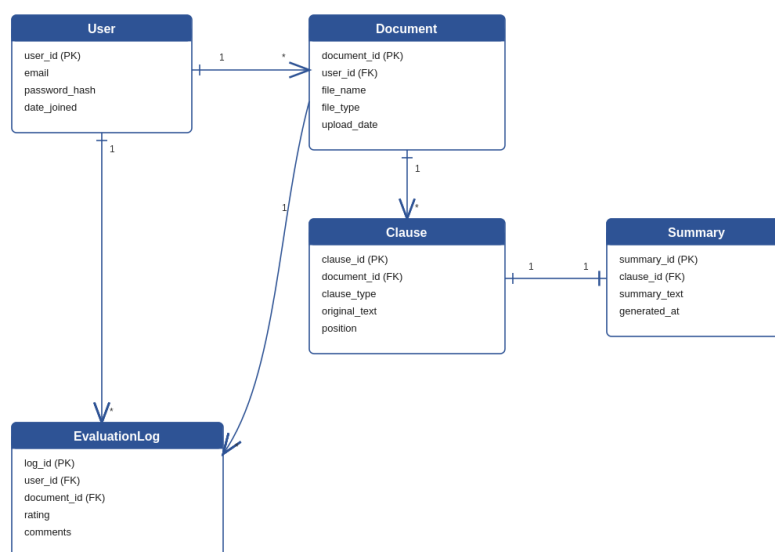


Figure 4.1: Entity Relationship Diagram of the Proposed System

Figure 4.1: Entity Relationship Diagram of the Proposed System

4.4.4 Program Module Specification

The application layer is decomposed into the following program modules, consistent with the System Architecture Diagram presented in Chapter Three (Figure 3.3).

The AuthenticationService module handles registration, login, token issuance and token validation using SimpleJWT. The FileProcessor module validates uploaded files and extracts raw text using PyPDF2 and python-docx. The TextPreprocessor module performs tokenization, sentence segmentation and named entity recognition using spaCy. The SummarizationEngine module generates abstractive plain-language summaries of identified clauses using Hugging Face Transformers. The APIRouter module exposes REST endpoints through Django REST Framework and routes requests to the appropriate module. The UserManager module manages user profile data and links uploaded documents to the correct account.

4.4.5 Input/Output Format

The system accepts a single input type from the user: a contract document in PDF or DOCX format, not exceeding a defined maximum file size enforced by the FileProcessor module. No other manual input is required from the user beyond authentication credentials during login.

The system produces a structured output for each processed document, returned to the presentation layer as a JSON object containing the document metadata, a list of identified clauses with their type labels, the original clause text, the corresponding plain-language summary for each clause, and a set of derived values used to render the timeline and summary-card visualizations, such as contract start date, end date and renewal date where these are present in the document.

4.4.6 Algorithm

The clause extraction and summarization pipeline follows the sequence of steps set out below.

```
BEGIN
  INPUT contract_file
  VALIDATE file_type, file_size
  raw_text <- EXTRACT_TEXT(contract_file)
  tokens <- SPACY_PREPROCESS(raw_text)
  clauses <- IDENTIFY_CLAUSES(tokens)
  FOR EACH clause IN clauses
    summary <- TRANSFORMER_SUMMARIZE(clause.text)
    clause.summary <- summary
    clause.type <- CLASSIFY_CLAUSE(clause.text)
  END FOR
  STORE clauses, summaries IN database
  RETURN structured_output TO presentation layer
END
```

4.4.7 Data Dictionary

Field	Table	Data Type	Description
user_id	User	Integer (PK)	Unique identifier for a registered user
email	User	Varchar(255)	User's login email address
password_hash	User	Varchar(255)	Hashed user password
document_id	Document	Integer (PK)	Unique identifier for an uploaded document
user_id	Document	Integer (FK)	Reference to the uploading user
file_name	Document	Varchar(255)	Original name of the uploaded file
file_type	Document	Varchar(10)	PDF or DOCX
upload_date	Document	Timestamp	Date and time of upload
clause_id	Clause	Integer (PK)	Unique identifier for an extracted clause
document_id	Clause	Integer (FK)	Reference to the parent document
clause_type	Clause	Varchar(50)	Category label, e.g. termination, payment
original_text	Clause	Text	Original clause text as extracted
summary_id	Summary	Integer (PK)	Unique identifier for a generated summary
clause_id	Summary	Integer (FK)	Reference to the parent clause
summary_text	Summary	Text	Plain-language summary of the clause

4.5 System Flowchart

The system flowchart in Figure 4.2 traces the execution path of the system from the point of user login through to the display of visualized results. Execution begins at user login, proceeds through document upload and validation, branches to an error path if the uploaded file fails validation, and otherwise continues through text extraction, clause identification,

summary generation and storage, before terminating at the display of the summary and visualization interface to the user.

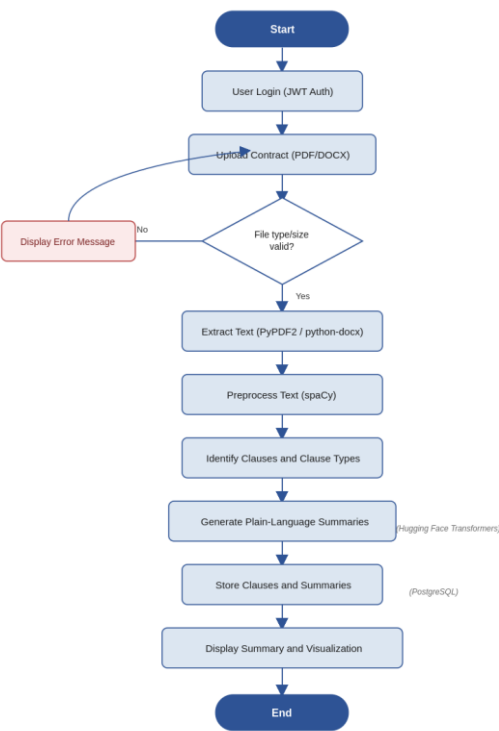


Figure 4.2: System Flowchart of the Proposed System

Figure 4.2: System Flowchart of the Proposed System

4.6 Object Diagrams

Figure 4.3 presents an object diagram illustrating a representative runtime instance of the system during the processing of a single uploaded document. The diagram shows a User object linked to a Document object, which in turn is linked to multiple Clause objects, each of which holds a reference to exactly one Summary object. This runtime structure mirrors the database relationships described in Section 4.4.2 and confirms that the logical design is preserved at the object level during execution.

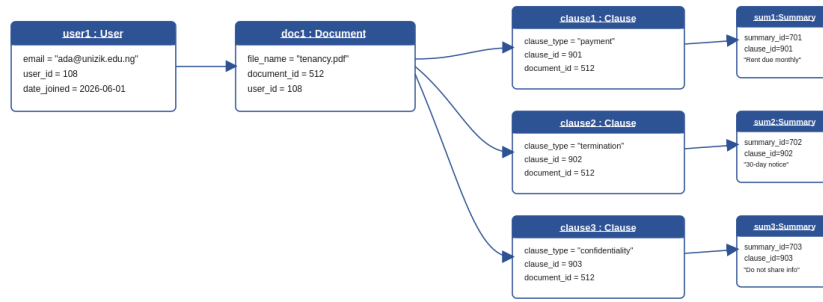


Figure 4.3: Object Diagram Showing a Runtime Instance of Document Processing

Figure 4.3: Object Diagram Showing a Runtime Instance of Document Processing

4.7 System Implementation

4.7.1 System Requirements

4.7.1.1 Hardware Requirements

The system requires a computer or server with a minimum of 8GB RAM for running the NLP models during development and testing, a minimum of 10GB of free storage for the application code, dependencies and the Hugging Face Transformer model files, and a stable internet connection for downloading pretrained models and for end users to access the deployed web application.

4.7.1.2 Software Requirements

The software requirements include an operating system capable of running Python 3.10 or later and Node.js, such as Windows, Linux or macOS. The backend requires Python 3.10 or later, Django, Django REST Framework, SimpleJWT, spaCy, Hugging Face Transformers, PyPDF2 and python-docx. The frontend requires Node.js, Next.js, Tailwind CSS and Recharts. The database requires PostgreSQL version 14 or later. Development tools used include Visual Studio Code as the Integrated Development Environment, Git and GitHub for version control, and Postman for API testing.

4.7.2 Program Development

4.7.2.1 Choice of Programming Environment

The backend of the system was developed using Python within the Django framework, chosen for its mature Object-Relational Mapper, built-in administrative interface and strong ecosystem support for integrating machine learning libraries such as spaCy and Hugging Face Transformers. The frontend was developed using Next.js, a React-based framework, chosen for its support for both client-side and server-side rendering, which improves the responsiveness of the upload and visualization interfaces. Tailwind CSS was used for styling because it allows rapid, consistent interface development without writing extensive custom CSS. Visual Studio Code was used as the primary Integrated Development Environment throughout development, and Git with GitHub was used for version control across the four Agile sprints described in Chapter Three.

4.7.3 Training

Because the system is intended for direct use by students and graduates without any technical background, no formal training programme is required for end users. The interface was designed to be self-explanatory, following the usability heuristics discussed in Chapter Two. Where users required guidance during the evaluation phase described in Chapter Five, a brief walkthrough of the upload and summary review process was provided.

4.7.4 Documentation

User-facing documentation consists of a short in-application guide accessible from the Dashboard, explaining how to upload a document and interpret the summary and visualization output. Technical documentation, intended for future maintenance of the system, includes inline code comments across all program modules, a README file describing the setup and installation process for both the Django backend and the Next.js frontend, and the data dictionary presented in Section 4.4.7.

4.7.5 System Conversion

4.7.5.1 Changeover Procedures

As established in Chapter Three, the existing process for contract review among Nigerian students and young adults is entirely manual, with no dedicated digital tool in prior use. Consequently, this project does not involve migrating data or users away from a pre-existing digital system. The changeover is therefore a direct introduction of the proposed system as a new, standalone tool, made available to users through a web browser without requiring any local installation.

4.7.5.2 Recommended Procedure

It is recommended that the system be introduced gradually, beginning with a pilot group of students for the evaluation phase described in Chapter Five, before wider release. This phased approach allows any issues identified during initial use, particularly relating to summarization accuracy or interface usability, to be addressed before the system is made available to a broader user base.